Backend

- **What is Backend ?**
  Backend in its traditional definition is a computer which is listening for HTTP or websocket or GRPC or any other kind of request through an open port which is accessible over internet so that clients or other frontend can connect to it , send data to it or receive data depending on the kind of request . We call it server cause it provides or it serves some kind of content whether it is static files (eg. Images or JS/HTML/JSON files) and it also accepts data if the client send something.

- **Parts of URL:**
  Let's consider an URL :
  blog.example.com
  - Subdomain: blog(The prefix added before the main domain to host a specific section or service)
  - Second-level domain: example(The unique name of the website)
  - Top-level domain: .com(The extension representing the root category or country
  - Full domain name : example.com(Combining the second level domain and top-level domain)
- **All about DNS :**
  DNS (Domain Name System) : DNS is the process that maps human friendly names to IP addresses . Without it we would have to memorize IP addresses .
  If local DNS server apparatus doesn't work or contain the needed record it sends the request up the DNS chain until the positive response is received back and then passed down back to original requester .
  For example: www.google.com
  www → specific service requested
  google → The local domain containing specific service
  .com → The top level that contains the specific service
  This is **FQDN(fully qualified domain name)**
  - **Different levels of DNS servers:**
    - Local DNS server: Server on local network contains hosts' files maps FQDN to IP in local subdomain
    - Top level Domain (TLD) server: Contains records for top level domain(.com,.edu,.org,.gov,.in)TLD servers do delegate down to second level servers to ease the load .
    - Root servers: Server containing records for the TLD servers.
  - **DNS records** :
    A record → Points to particular IP
    C record → Points to a particular domain name or sub domain
    A record → Maps the hostnames to their IPv4 address
    AAAA record →Maps the hostnames to their IPv6 address
    NAME record → Maps the canonical names(alias name) to host names
    PTR record → Pointer record that points to a canonical name
    MX record → Maps to email server that is specified for a specific domain . Record that determines how email travels from sender to recipient .

- **Types of Responses :**
  - Authoritative: A auth DNS server is the one that responds to a request that has been specifically configured to contain information , that actually holds the original records
  - Non – authoritative: A non-auth DNS server is one that responds to a request with DNS information that is received from another DNS server . A non auth response is not a response from official name server for the domain ,instead a second or third had response.

Request searches to the host through DNS and IP and before it reaches the server or particular computer it goes through a firewall.

- **Security groups in AWS**

EC2 is a virtual server used to host websites and application , run backend databases handle high performance computing . Security groups in AWS allows it is assigned to our AWS instance using this we can specify which ports we want to allow and which ports we want accessible to the internet

SSH port is generally used by the developer to log into AWS instance to terminals or cmd so that we can do different types of operations .

Request → Domain server → firewall → AWS instance → Nginx→Local Host

If HTTP and HTTPS ports(80 and 443 respectively) are blocked then the requests wont be able to reach our server.

- **Reverse Proxy:**

A server which sits in front of other server so that we can manage different types of redirects or configs from a centralized place instead of a changing configs in every single server for that Nginx is used. Request that comes to domain Nginx config redirects those requests to local host where the server is running on. Nginx and domain names are used to route requests to our local server over internet.

- **Need for backend:**

Let's take an example when we like friend's post on Instagram: The app sends a request to the server parses the request gets the user and it finds name/id etc. and if persists, saves the data (the action of your liking the post) in a way, usually in a database and then it checks the user (the friend's post's id) id and sends notification it triggers some kind of action that sends notification to the user and user gets the notification. Server has to be centralised.

- **How frontend works :**

Browser fetches HTML file and all resources and fetched in different different requests after primary HTML file in DNS A record we can see the entry of frontend ,demo,subdomain and it's IP and it is connected to EC2 instance HTTPS and HTTP ports has to be allowed so traffic gets through server .Whatever frontend logic that is written is all fetched by the browser from the server and it is executed by the browser in client's machine .

Browser runtimes are sandbox runtimes isolated from OS so the code can only access limited amount of resources (document object , browser APIs or cookies and external APIs) . There are many ways to get around the problem caused by CORS (CORS : Security policy of browsers which restricts JS code to call external APIs which is not the same as current domain) and use of HTTP headers allow it to use external APIs that don't share the same domain

Why CORS and Sandboxing of the browser:

Essentially browser is fetching code from remote server executing it in users browser and if its not careful and isolated enough the remote code can easily access data or files from users computer and if can copy all files sensitive details and send it to their servers.

Why can't we write backend logic in frontend?

1. Security Reasons : Security policies of browsers are restrictive and sometimes backend needs to access underlying file system whether it's to write a log file or access environment variales and browser wont allow that
2. Cannot call external APIs: External APIs can't be called due to CORS unless APIs have CORS headers.
3. Databases: Server runtime has access to all native database drivers and that allows it to communicate efficiently with databases these drivers are written to working environments that can handle socket connections handle binary data and maintain persistent connection , things that browsers cannot do.
4. Computing Power: User might not have enough computing power to perform heavy business logic and things will start to lag and break because of load, so centralized backend server helps increase CPU and memory whenever we want.

** Backend servers maintain a list of connection called connection pool to our database server so that it doesn't have to create and destroy connections again and again